Service Agent-Transport Agent Task Planning Incorporating Robust Scheduling Techniques

Matthew J. Bays[a,1,3,*], Thomas A. Wettergren[b,2]

[a]*Panama City, FL, 32407*
[b]*Newport, RI 02841*

Abstract

As the use of cooperative, heterogeneous teams of autonomous robots to perform tasks such as autonomous package delivery and long-duration ocean sampling becomes more prevalent, the is a quickly-emerging need to study the high-level interaction of specialized robotic agents that perform service tasks, and specialized transport agents that transport the service agents between service areas. If the service routes, docking, and deployment schedules are not carefully planned, the overall schedule is inefficient at best, and possibly even infeasible due to fuel limitations at worst. We introduce a new problem in the area of scheduling and route planning operations called the service agent transport problem (SATP). Within the SATP, autonomous service agents must perform tasks at a number of locations. The agents are free to move between locations, however, the agents may also be transported throughout the region by a limited number of faster-moving transport agents. The goal of the SATP is to plan a schedule of service agent and transport agent actions such that all locations are serviced in the shortest amount of time. We believe the SATP formulation is unique because there is strong coupling between vehicle constraints as well as between the task allocation component of the problem and the scheduling component of the problem. We present a solution to the problem using a mixed-integer linear programming

*Principal corresponding author
Email addresses: matthew.bays@navy.mil (Matthew J. Bays), thomas.wettegren@navy.mil (Thomas A. Wettergren)
[1]Naval Surface Warfare Center, Panama City Division
[2]Naval Undersea Warfare Center, Newport Division
[3]Tel: 850-230-7711

optimization framework and compare several complexity reduction heuristics to full optimization. Additionally, we include methods to account for relative uncertainty in the duration of planned tasks in such a manner as to balance the risk of schedule slips (conflicts) to the risk of creating an overly conservative and sub-optimal schedule.

Keywords:
Autonomous Agents, Robotics, Scheduling, Bayes Risk, Robust Scheduling

1. Introduction

The use of autonomous systems to perform increasingly complex and coordinated tasks has necessitated creating heterogeneous teams of agents, where different agent types specialize in different parts of an operation [1, 2, 3]. One such heterogeneous team operation is when one type of mobile agent is tasked with performing the direct servicing tasks for the operation, while another type of larger, faster-moving, or longer-range agent is responsible for transporting the service agents between jobs for faster completion. In this work we call the two types of agents *service agents* and *transport agents*, respectively. Within manned systems, there are numerous examples of the transport agent/service agent concept such as aircraft carriers and their respective aircraft, garbage trucks and accompanying garbage workers, or mail delivery vehicles and their respective postmen. While this form of close interaction between unmanned systems is still far from common, the underlying hardware and guidance infrastructure to allow autonomous docking and deployment between unmanned systems are being researched for a variety of different applications [4, 5, 6, 7, 8].

Just as important as the fundamental infrastructure of docking and deploying unmanned systems autonomously is determining the most efficient schedule for *when* and *where* to perform these actions when multiple agents can be assigned to perform an operation. Indeed, new methods of package delivery such as cooperative teams of aerial drones and shipping trucks are being explored by leading technology companies [9]. Furthermore, coopera-

This paper was presented in part at the *IEEE International Conference on Intelligent Robots and Systems*, Hamburg, Germany September 2015.

tive teams of unmanned underwater vehicles (UUVs) and unmanned surface vessels (USVs) are quickly emerging where one unit performs the substantive survey operations while the other is used for transportation and refueling [10]. We seek to design planning algorithms to solve the necessary task allocation, planning, and scheduling necessary to ensure the service agents such as drones and UUVs can accomplish their tasks safely and efficiently, while the transport agents perform the necessary transport and refueling operations.

The primary contribution of our work is a formal mathematical framework for planning a schedule of service agent and transport agent actions such that all locations are serviced in the shortest amount of time. We call the framework the service agent transport problem (SATP). In addition to defining the mathematical framework, we show that the problem is NP-hard, and propose several methods to limit the search space for use with mixed-integer linear programming (MILP) techniques. The SATP is a novel general problem in multi-agent planning and scheduling, and proving complexity demonstrates that future research should emphasize effective and computationally efficient optimization heuristics. We next extend the framework to handle uncertainty in task duration by determining optimal buffer times between tasks such that a cost function inspired by Bayes risk is minimized. The result of our work is both a formal analysis of the high-level planning algorithms necessary for service agent and transport agent teams such as aerial drones and delivery trucks or UUVs and USVs, as well as an initial computational framework for solving the problem using a combination of MILP and complexity reduction heuristics. Thus, we provide a basis that will enable formal agent plans to be developed for efficient, safe, and robust schedules of service agent and transport agent interactions.

The SATP is different from other related planning and scheduling problems in two significant ways: strong cross-schedule coupling between agent constraints, and strong coupling between the scheduling problem and task allocation problem. By strong cross-schedule constraints, we mean that the schedule of individual agent actions, such as docking and deployment, are highly dependent on the location and schedule of multiple agents: in order for a service agent to dock with a transport agent, both agents must be located at the same position and untasked for a certain segment of time. We consider task allocation and scheduling decoupled in a problem when the decision as to *whether* a task is to be executed or what agent is to execute the task is determined separately from *when* that task is to be executed. Indeed, in the majority of scheduling work we have reviewed, see for example

[11, 12, 13, 14], it is implicitly assumed that there is some list of tasks that must be executed, and the scheduling problem is to determine which asset and in what order to perform those tasks, subject to a set of constraints. In our work, we do not assume that specific lower-level tasks such as docking, deployment, or movement will be executed by any one agent. Instead, the non-trivial constraints we propose as part of the SATP mandated locational requirements for accomplishing the high-level service tasks, and the optimization determines the best specific subtasks to execute in order to satisfy the locational constraints.

1.1. Related Work

The problem of scheduling and task allocation has been extensively studied with a variety of techniques in multiple forms. Koes et al. developed a coordination architecture for modeling multirobot coordination and task allocation in [15]. Our framework complements this methodology by expanding the types of scenarios under which a constraint optimization framework can be used: namely to problems where there are strongly coupled constraints on agents such as transportation actions.

Korsah et al. create a framework to explore optimal vehicle routing with cross-schedule constraints with applications to robotic assistance in [16]. However, there are several notable differences to our work. Firstly, the principal element of optimization is an agent *route*, not the collection of agent actions comprising the route. Additionally in Korsah's route optimization problem, the individuals being transported along the routes are static; there is no mechanism within the optimization framework for the individuals themselves to move about the area. As such [16] is similar to the traditional vehicle routing problem (VRP) [17]. Mathew et al. addresses finding the shortest route for an unmanned ground vehicle (UGV) and unmanned aerial vehicle (UAV) pair to transit and make deliveries to locations only reachable by the UAV in [18]. Similarly, Garone et al formulate a problem involving an aircraft carrier and aerial vehicle [19]. However, in both their works, the authors focuse primarily on the overall path planning for the two vehicles, and not on fuel limitations and the higher-level scheduling aspects required when there are an arbitrary number of UAVs and UGVs. Our work takes a scheduling-centric approach, formally incorporates fuel constraints, and is generalized for an arbitrary number of service agents and transport agents.

Gombolay et al. develop a centralized algorithm to efficiently schedule manufacturing processes using robotic teams in [13]. However, they primarily

focus on the development of a novel task sequencing heuristic that allows the generalized problem, a Simple Temporal Problem [20], coupled with simple spatio-temporal constraints, to be tractable for large numbers of tasks and robots to be assigned to a fixed set of tasks. Our principal contribution, in contrast, is in the formal definition of the novel simple agent transport problem, unique modeling and constraint-based logic required for the docking, transport, and deployment of service agents throughout the environment, and resulting analysis. While their algorithm schedules deterministic problems, our work handles schedule uncertainty for the agents. In our work tasks to be completed consist of a set of locations, and the agents must service each location set. However, to do so, the agents may choose from a number of different docking, deployment, and movement actions to allow a location to be serviced. The chosen actions also have cross-schedule constraints with the transport agents.

General robust scheduling techniques have been studied using a variety of methods. Lin, Janak, and Floudas develop complimentary approaches to formally incorporate uncertainty in scheduling problems in [12] and [21]. In their work, Lin et al. develop formal methodologies for converting MILP scheduling problems where model parameters are uncertain under either a bound or known probability distribution into another MILP or MINLP-type problem where the uncertainty is explicitly incorporated into the optimization framework, yielding a robust solution that still meets the problem constraints with a fixed probability. In a similar manner, Bertsimas and Sim create a method to adjust the conservatism of robust MILP solutions in [22] while keeping the robust problem linear. Other work on robust scheduling and integer programming problems in the general sense may be found in [23, 24, 25]. However, to our knowledge there is no work explicitly incorporating Bayes risk to schedule slips nor for the application of service agent - transport agent scheduling. Our work has the similar goal for the specific problem of service agent - transport agent scheduling, but instead of robustly optimizing with a fixed uncertainty of meeting time constraints or an acceptable number of constraints that may be violated, we incorporate a cost function derived from Bayes risk using an adjustable cost of a schedule slip and the potential increase in buffer time for a task. Bayes risk is a compelling tool for optimizing decisions under uncertainty because it formally and rigorously incorporates both cost and likelihood of making a particular decision.

This paper is outlined as follows: in Section 2, we discuss the general setup of the service agent transport problem. In Section 3, we outline the mathe-

matical constraints that make up the SATP framework. We propose several heuristics in order to limit the computational complexity in 4. We discuss robust scheduling extensions that utilize a cost function based on Bayes risk which adds buffer times in between tasks into the system in Section 5. Finally, simulation results showing the performance of various implementations of the SATP are discussed in Section 6.

2. Problem Definition

Let there exist a set of autonomous agents $\mathcal{A} = \{1, \ldots, A\}$ that are tasked with servicing a number of service areas $\mathcal{S} = \{1, \ldots, S\}$. The service areas are connected by a directed graph $\langle \mathcal{D}, \mathcal{E} \rangle$. The nodes $\mathcal{D} = \{o, 1, \ldots, D\}$ represent locations at which an agent $a \in \mathcal{A}$ may enter a service area $s \in \mathcal{S}$ in order to perform operations. The node labeled o is the origin node, which represents the starting point for all valid paths in the graph. The agents may move from node i to node j along an edge of the graph $e = (i, j) \in \mathcal{E}$. The subset $\mathcal{D}_s \subset \mathcal{D}$ represents nodes from which agents may service area s.

The use of graph representations of scheduling problems has been historically used effectively due to the ability of the methods to directly lead to effective heuristic decomposition procedures [26] that can greatly reduce the computational complexity of the optimization. Structural simplifications of the graph structure may be used to create heuristics on the service scheduling; for example, the resulting optimized schedule is expected to be from the subset of acyclic paths within the graph structure. Thus, reducing the problem size by decomposing the graph structure into a directed acyclic graph structure is a reasonable complexity reduction strategy; such an approach is considered in Section 4.

The agents are separated into two mutually exclusive and collectively exhaustive subsets $\mathcal{A}_S$ and $\mathcal{A}_T$ consisting of service agents and transport agents, respectively. A service agent $m \in \mathcal{A}_S$ is capable of directly servicing the service areas, while a transport agent $n \in \mathcal{A}_T$ may collect a fixed number of the service agents and transport them between nodes. The service agents may move between nodes with a time cost c_{ij}^{move}. However, they are significantly slower than transport agents with transportation cost of $c_{ij}^{trans} \leq c_{ij}^{\text{move}}$. The transport agents also require a fixed amount of time to collect and deploy the service agents. The collection and deployment costs are denoted c^{dock} and c^{deploy}, respectively. The agents initially start at an origin node o, with each transport agent n containing a fixed number of service agents such that all

service agents start docked with a transport agent. Figure 1 shows a notional illustration of the service agent transport problem.

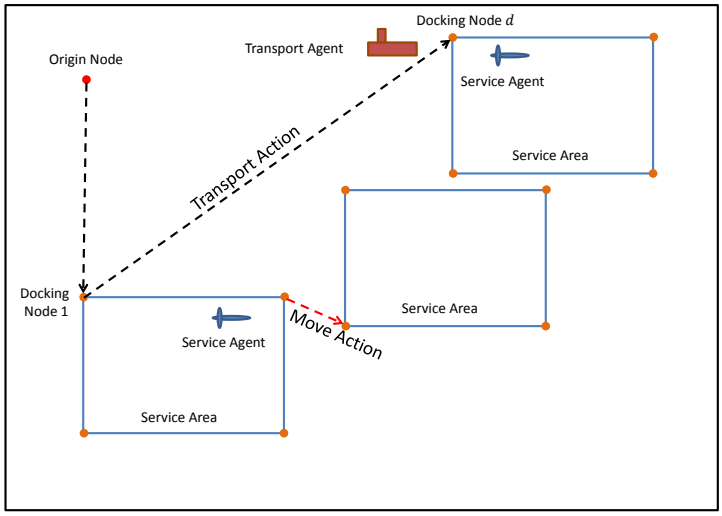


Figure 1: Notional illustration of service agent transport problem.

We propose a phase-based model for the problem in that every agent may perform a single task during a *phase* of the mission, with phases labeled $\mathcal{P} = \{1, \ldots, P\}$. Each agent-phase pair can have a varying time length that is dependent on the particular agent and action scheduled to be performed during the phase. During a phase, an agent may perform at most one task from the set of tasks $\mathcal{K} = \{\text{move}, \text{service}, \text{dock}, \text{deploy}\}$. When service agents perform a move action, they only move themselves, while transport agents may simultaneously move up to a specified number of service agents currently docked with the transport agent.

Given the above terminology, the phase-based optimization problem can be formulated as follows:

Problem 2.1. *SATP - Find a sequence of service agent and transport agent tasks* $\bar{\mathbf{I}}$ *and start times* $\bar{\mathbf{T}}^{\text{start}}$ *during phases* $1, \ldots, P$ *such that all areas* $\mathcal{S}$ *are serviced, fuel limitations are observed during each phase, and all service agents are docked with a transport agent at the end of phase* P *such that* $\max_{a \in \mathcal{A}} \bar{\mathbf{T}}^{\text{end}}_{a,P}$ *is minimized.*

The primary optimization variables for our specific solution to the SATP, which we call *decision variables*, fall into two groups: scheduling indicator variables and time variables. The scheduling indicator variable $\bar{\mathbf{I}}^{k}_{a,p,\mathcal{L}} \in \{0,1\}$ denotes that task type k is scheduled for completion by agent a during phase p at location $l \in \mathcal{L}_k$, where $\mathcal{L}_k$ is the set of all feasible locations required for execution of task type k and $\bigcup_{k\in\mathcal{K}} \mathcal{L}_k = \mathcal{L}$. The timing variables $\bar{\mathbf{T}}^{\text{start}}_{a,p} \in \mathbb{R}$ and $\bar{\mathbf{T}}^{\text{end}}_{a,p} \in \mathbb{R}$ denote the times at which the task of agent a during phase p is scheduled to start and end, respectively. In contrast to the decision variables, additional variables are needed for the optimization problem in order to develop the necessary constraints to formulate the optimization problem. While the additional variables, which we denote as *helper variables* are technically variables used in the optimization, however are not truely independent variables in a problem formulation sense, but instead are constrained optimization variables used to create formal constraints. We will discuss the helper variables as the details of the optimization problem are formulated.

3. Constraint Formulation

We now turn to defining the constraints required for the mathematical formulation of the service agent transport problem.

3.1. Docking and Deployment Constraints

The first constraint within the SATP is that a service agent must only complete certain tasks when they are docked or deployed, as appropriate. For enforcing this constraint, we create the helper variables to track whether a service agent m is docked or deployed from a transport agent n. We denote this variable

$$\bar{\mathbf{D}}_{mn,p} = \begin{cases} 1 & \text{Agent } m \text{ is docked with agent } n \text{ in phase } p \\ 0 & \text{Otherwise} \end{cases} \tag{1}$$

Within an integer programming framework, we create (1) with the formal constraint

$$\begin{aligned} &\forall m \in \mathcal{A}_S, \forall n \in \mathcal{A}_T, \forall p \in \mathcal{P} \\ &\bar{\mathbf{D}}_{mn,p} = \sum_{p' \le p, d \in \mathcal{D}} \left[\bar{\mathbf{I}}^{\text{dock}}_{mn,p',d} - \bar{\mathbf{I}}^{\text{deploy}}_{mn,p',d} \right], \end{aligned} \tag{2}$$

where $\bar{\mathbf{I}}^{\text{dock}}_{mn,p',d}$ and $\bar{\mathbf{I}}^{\text{deploy}}_{mn,p',d}$, are indicator variable docking and deploying, respectively, at particular dock/deploy nodes d. We bound $\bar{\mathbf{D}}_{mn,p}$ with the constraint

$$\forall m \in \mathcal{A}_S, \forall n \in \mathcal{A}_T, \forall p \in \mathcal{P} \qquad 0 \leq \bar{\mathbf{D}}_{mn,p} \leq 1. \tag{3}$$

Constraints (2) and (3) now causes $\bar{\mathbf{D}}_{mn,p}$ to equal 1 if agent m is currently docked with agent n during phase p, and 0 otherwise. Constraint (2) and the lower bound in (3) satisfies the additional requirement that in order for a service agent to be deployed, it must first have been docked. It is also necessary to track if an agent is docked or deployed in general. This is performed using the helper variable $\bar{\mathbf{D}}_{mn,p}$ and constraints

$$\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad \bar{\mathbf{D}}_{m,p} = \sum_{n \in \mathcal{A}_T} \bar{\mathbf{D}}_{mn,p} \tag{4}$$

and

$$\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad 0 \leq \bar{\mathbf{D}}_{m,p} \leq 1 \tag{5}$$

$$\tag{6}$$

Additionally, we limit the total number of service agents that can be docked at a given time by a transport agent to D_{max} using the constraint

$$\forall n \in \mathcal{A}_T, \forall p \in \mathcal{P} \qquad \sum_{m \in \mathcal{A}_S} \bar{\mathbf{D}}_{mn,p} \leq D_{max} \tag{7}$$

With constraints (4)-(7), we can now limit the feasible actions of the individual agents when deployed or docked as appropriate.

3.2. Fuel Limitation and Charging Constraints

Fuel consumption is an additional variable that must be tracked for each agent during all phases of the mission. For tracking fuel, we define the variable $\bar{\mathbf{F}}_{m,p} \in \mathbb{R}$ as the amount of fuel service agent m has available during the

start of phase P. The variable $\bar{\mathbf{F}}_{m,p}$ is assigned a value using the constraints

$$\begin{aligned} &\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad (8)\\ &\bar{\mathbf{F}}_{m,p} = \sum_{p' \leq p} \Bigg[\bar{\mathbf{F}}^{charge}_{m,p'} - \sum_{s \in \mathcal{S}} f^{\text{service}}_{m,s} \bar{\mathbf{I}}^{\text{service}}_{m,p',s} \\ &- \sum_{d \in \mathcal{D}} \left(f^{\text{dock}}_{m,d} \bar{\mathbf{I}}^{\text{dock}}_{m,p',d} + f^{\text{deploy}}_{m,d} \bar{\mathbf{I}}^{\text{deploy}}_{m,p',d} \right) \\ &- \sum_{v \in \mathcal{E}} f^{\text{move}}_{m,p',v} \bar{\mathbf{I}}^{\text{move}}_{m,p',v} \Bigg] \end{aligned}$$

where $f^{\text{service}}_{m,s}$, $f^{\text{dock}}_{m,d}$, $f^{\text{deploy}}_{m,d}$, and $f^{\text{move}}_{m,v}$ are fixed costs for executing their respective goals, and $\bar{\mathbf{F}}^{charge}_{m,p}$ is a floating variable indicating the amount an agent is charged during a phase. The variable $\bar{\mathbf{F}}^{charge}_{m,p}$ is fixed to its value using the linear service agent charge rate c^{charge}_{m} and the constraints

$$\begin{aligned} &\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad (9)\\ &\bar{\mathbf{D}}_{m,p} = 1 \Rightarrow \bar{\mathbf{F}}^{charge}_{m,p} \leq c^{charge}_{m} \left(\bar{\mathbf{T}}^{\text{end}}_{m,p} - \bar{\mathbf{T}}^{\text{start}}_{m,p} \right) \\ &\bar{\mathbf{D}}_{m,p} = 0 \Rightarrow \bar{\mathbf{F}}^{charge}_{m,p} = 0. \end{aligned}$$

Additionally, we limit the fuel capacity with the constraints

$$\begin{aligned} &\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad (10)\\ &\bar{\mathbf{F}}_{m,p} \leq F_{max} \\ &\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad (11)\\ &0 < \bar{\mathbf{F}}_{m,p} \end{aligned}$$

3.3. Goal and Capability Constraints

In our formulation, each agent can only prosecute one goal during a phase of the mission. As such, we have the constraint

$$\forall a \in \mathcal{A}, \forall p \in \mathcal{P} \qquad \sum_{k \in \mathcal{K}} \sum_{l \in \mathcal{L}_k} \bar{\mathbf{I}}^{k}_{a,p,l} \leq 1. \qquad (12)$$

As not every agent is capable of completing every task, this requires additional constraints to force only agents capable of completing a task to be scheduled for the task. As such, we have the constraint

$$\forall a \notin \mathcal{A}_k, \forall p \in \mathcal{P}, \forall l \in \mathcal{L}_k \qquad \bar{\mathbf{I}}^{k}_{a,p,l} = 0 \qquad (13)$$

We note that this constraint may also be implemented for tractability by limiting both the scheduling variables $\bar{\mathbf{I}}^k_{a,p,l}$ and constraints specific to a certain capability that are created in the optimization problem to those allowed by the individual agent's capabilities.

3.4. Time Dependency and Wait Constraints

We now turn to the constraints required for determining the specific times at which phases are scheduled for starting and ending. Without loss of generality, we assume that the start time of the initial phase is at $t = 0$. As such we have the constraint

$$\forall a \in \mathcal{A} \qquad \bar{\mathbf{T}}^{\text{start}}_{a,p=0} = 0. \tag{14}$$

In order to adhere to the boundary conditions between the end of the last phase and the start of the next phase, we have the constraint

$$\forall a \in A, \forall p \in \mathcal{P} \,|\, p \neq 0 \qquad \bar{\mathbf{T}}^{\text{start}}_{a,p+1} \geq \bar{\mathbf{T}}^{\text{end}}_{a,p} \tag{15}$$

With constraints (14) and (15), we have defined all variables with the exception of the end time of each phase, $\bar{\mathbf{T}}^{\text{end}}_{a,p}$. The end time may be defined by taking advantage of (12) with only one task being scheduled for an agent during a phase

$$\forall a \in \mathcal{A}, \forall p \in \mathcal{P} \qquad \bar{\mathbf{T}}^{\text{end}}_{a,p} = \bar{\mathbf{T}}^{\text{start}}_{a,p} + \sum_{k \in \mathcal{K}} \sum_{l \in \mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p,l} \tag{16}$$

where $c^k_{a,L}$ is the duration of task k if performed by agent a on location set L. Additionally, the dual relationship between a service agent and transport agent performing docking and deployment constraints necessitates that we must ensure that those particular operations occur at the same time for both agents. Thus, we have the time dependency constraints

$$\forall m \in \mathcal{A}_S, \forall m \in \mathcal{A}_T, \forall p \in \mathcal{P} \tag{17}$$
$$\sum_{d \in \mathcal{D}} \bar{\mathbf{I}}^{\text{dock}}_{mn,p,d} = 1 \Rightarrow \bar{\mathbf{T}}^{start}_{m,p} = \bar{\mathbf{T}}^{\text{start}}_{n,p}$$

$$\forall m \in \mathcal{A}_S, \forall m \in \mathcal{A}_T, \forall p \in \mathcal{P} \tag{18}$$
$$\sum_{d \in \mathcal{D}} \bar{\mathbf{I}}^{\text{deploy}}_{mn,p,d} = 1 \Rightarrow \bar{\mathbf{T}}^{\text{end}}_{m,p} = \bar{\mathbf{T}}^{\text{end}}_{n,p}$$

Some scheduling situations necessitate an agent loitering or otherwise performing no meaningful actions during a given phase so that it may properly coordinate with the actions of other agents. For example, a transport agent may need to wait at a docking point during a phase in which a service agent is servicing an area with which it will dock in the proceeding phase. In this case, we require an additional variable indicating that an agent is idle during that phase at a given location. We define the $\bar{\mathbf{I}}^{wait}_{a,p,d}$ indicator variable by the constraints

$$\forall a \in \mathcal{A}, \forall p \in \mathcal{P}, \forall d \in \mathcal{D} \tag{19}$$
$$\sum_{k \in \mathcal{K}} \left[\sum_{l \in \mathcal{L}_k} \bar{\mathbf{I}}^{k}_{a,p,l}] \right] + \sum_{d' \in \mathcal{D} | d' \neq d} \bar{\mathbf{I}}^{wait}_{a,p,d'} = 1 \Rightarrow \bar{\mathbf{I}}^{wait}_{a,p,d} = 0$$

$$\forall a \in \mathcal{A}, \forall k \in \mathcal{K}, \forall p \in \mathcal{P}, \forall d \in \mathcal{D} \tag{20}$$
$$\sum_{k \in \mathcal{K}} \left[\sum_{\substack{l \in \mathcal{L}_k | \\ d \notin \mathcal{L}}} \bar{\mathbf{I}}^{k}_{a,p-1,l} \right] + \sum_{\substack{d' \in \mathcal{D} | \\ d' \neq d}} \bar{\mathbf{I}}^{wait}_{a,p-1,d'} = 1 \Rightarrow \bar{\mathbf{I}}^{wait}_{a,p,d} = 0$$

$$\forall a \in \mathcal{A}, \forall p \in \mathcal{P} \tag{21}$$
$$\sum_{k \in \mathcal{K}} \sum_{l \in \mathcal{L}_k} \bar{\mathbf{I}}^{k}_{a,p,l} = 0 \Rightarrow \sum_{d \in \mathcal{D}} \bar{\mathbf{I}}^{wait}_{a,p,d} = 1.$$

Constraint (19) forces $\bar{\mathbf{I}}^{wait}_{a,p,d}$ to equal zero if the agent is otherwise tasked or waiting at another location. Constraint (20) forces $\bar{\mathbf{I}}^{wait}_{a,p,d}$ to equal zero if the agent performed an action at a different location d' during the previous time step, thereby prohibiting the agent from waiting at location d. Constraint (21) forces $\bar{\mathbf{I}}^{wait}_{a,p,d}$ to one if the agent is not otherwise tasked.

3.5. Transit and Transport Constraints

We now turn to the constraints required for transiting agents and transporting them. In order for an agent to transit from one location to the next,

it must have performed an action at the previous location

$$\begin{aligned}
&\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P}, \forall d_0 \in D \\
&\quad \sum_{d \in \mathcal{D}} \bar{\mathbf{I}}^{\text{move}}_{m,p,(d_0,d)} = 1 \\
&\quad \Rightarrow \sum_{d' \in \mathcal{D}} \bar{\mathbf{I}}^{\text{move}}_{m,p-1,(d',d_0)} + \sum_{n \in \mathcal{A}_T} \bar{\mathbf{I}}^{\text{deploy}}_{mn,p-1,d_0} \\
&\quad + \sum_{s \in \mathcal{S} | d_0 \in \mathcal{D}_s} \bar{\mathbf{I}}^{\text{service}}_{m,p-1,s} + \bar{\mathbf{I}}^{wait}_{m,p-1,d} = 1.
\end{aligned} \tag{22}$$

where $(d_0, d), (d', d_0) \in \mathcal{E}$. Additionally, we have the constraint that for a agent to either move from one area to the next or to service an area, the agent must be deployed

$$\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad \sum_{v \in \mathcal{E}} \bar{\mathbf{I}}^{\text{move}}_{m,p,v} = 1 \Rightarrow \bar{\mathbf{D}}_{m,p} = 0 \tag{23}$$

For an agent m to be docked with another agent n capable of capturing agents, both agents must have been located previously at the location of the capture. Thus, we have

$$\begin{aligned}
&\forall m \in \mathcal{A}_S, \forall n \in \mathcal{A}_T, \forall p \in \mathcal{P}, \forall d \in \mathcal{D} \\
&\quad \bar{\mathbf{I}}^{\text{dock}}_{mn,p,d} = 1 \Rightarrow \sum_{d_{0_1} \in D} \bar{\mathbf{I}}^{\text{move}}_{m,p-1,(d_{0_1},d)} + \bar{\mathbf{I}}^{wait}_{m,p-1,d} \\
&\quad + \sum_{s \in \mathcal{S} | d \in \mathcal{D}_s} \bar{\mathbf{I}}^{\text{service}}_{m,p-1,s} = 1
\end{aligned} \tag{24}$$

$$\begin{aligned}
&\bar{\mathbf{I}}^{\text{dock}}_{mn,p,d} = 1 \Rightarrow \sum_{d_{0_1} \in D} \bar{\mathbf{I}}^{\text{move}}_{n,p-1,(d_{0_1},d)} \\
&+ \bar{\mathbf{I}}^{wait}_{n,p-1,d} \\
&+ \sum_{\substack{m' \in \mathcal{A}_S | \\ m' \neq m}} \left[\bar{\mathbf{I}}^{\text{dock}}_{m'n,p-1,d} + \bar{\mathbf{I}}^{\text{deploy}}_{m'n,p-1,d} \right] \geq 1
\end{aligned} \tag{25}$$

where $(d_{0_1}, d), (d_{0_2}, d) \in \mathcal{E}$. Likewise, we have the constraint for deploying

agents as

$$\forall m \in \mathcal{A}_S, \forall n \in \mathcal{A}_T, \forall p \in \mathcal{P}, \forall d \in \mathcal{D} \tag{26}$$
$$\bar{\mathbf{I}}^{\text{deploy}}_{mn,p,d} = 1 \Rightarrow \bar{\mathbf{D}}_{mn,p} = 1$$
$$\bar{\mathbf{I}}^{\text{deploy}}_{mn,p,d} = 1 \Rightarrow \sum_{d_{0_1} \in D} \bar{\mathbf{I}}^{\text{move}}_{mn,p-1,(d_{0_1},d)} \tag{27}$$
$$+ \bar{\mathbf{I}}^{wait}_{n,p-1,d} + \sum_{\substack{m' \in \mathcal{A}_S| \\ m' \neq m}} \left[\bar{\mathbf{I}}^{\text{dock}}_{m'n,p-1,d} + \bar{\mathbf{I}}^{\text{deploy}}_{m'n,p-1,d}\right] \geq 1$$

3.6. Servicing Constraints

The first constraint for a servicing action is that a service area can only be serviced if it is scheduled after a deployment action at a node $d \in \mathcal{D}_s$ or move action. Thus, we have the constraint

$$\forall m \in \mathcal{A}_S, \forall n \in \mathcal{A}_T, \forall p \in \mathcal{P}, \forall d \in \mathcal{D}, \forall s \in \mathcal{S} \tag{28}$$
$$\bar{\mathbf{I}}^{\text{service}}_{m,p,s} = 1 \Rightarrow \sum_{n \in \mathcal{A}_T} \bar{\mathbf{I}}^{\text{deploy}}_{mn,p-1,d} + \sum_{d' \in D} \bar{\mathbf{I}}^{\text{move}}_{m,p-1,(d',d)} = 1.$$

where $(d', d) \in \mathcal{E}$ Next, we have the constraint that a service action can only be performed if it is scheduled while the agent is deployed

$$\forall m \in \mathcal{A}_S, \forall p \in \mathcal{P} \qquad \sum_{s \in \mathcal{S}} \bar{\mathbf{I}}^{\text{service}}_{m,p,s} = 1 \Rightarrow \bar{\mathbf{D}}_{m,p} = 0 \tag{29}$$

Additionally, every service goal should be completed exactly once by the group of agents. Thus,

$$\forall s \in \mathcal{S} \qquad \sum_{m \in \mathcal{A}_S} \sum_{p \in \mathcal{P}} \bar{\mathbf{I}}^{\text{service}}_{m,p,s} = 1. \tag{30}$$

3.7. Cost Function

The cost function to be minimized in our initial formulation is to minimize the end time of the last phase over all agents. That is, our objective is

$$\text{minimize } \max_{a \in \mathcal{A}} \bar{\mathbf{T}}^{\text{end}}_{a,P}. \tag{31}$$

4. Complexity and Complexity Reduction

Now that we have fully modeled the service agent transport problem, we now discuss initial methods to limit the search space required by MILP solvers. The full optimization is NP hard, which can be shown with the following proofs.

Proposition 4.1. *The SATP belongs to the class NP.*

Proof. We first pose the corresponding decision problem to the optimization problem constructed from constraints (2)-(30) and cost function (31) as a formal language.

$$\begin{aligned} SATP = \{&\langle \mathcal{D}, \mathcal{E} \rangle, \mathbf{c}, S, M, N, l, f(\cdot), P, T'\} \\ &f : \mathcal{D} \to \mathcal{S} \\ &T' \in \mathbb{R} \\ &M \text{ service agents and } N \text{ transport agents are} \\ &\text{assigned tasks during phases } \mathcal{P} = \{1, \ldots, P\} \\ &\text{such that constraints (2)-(30) hold with} \\ &\max_{p \in \mathcal{P}} \bar{\mathbf{T}}^{\text{end}}_{a,P} \text{ of at most } T'. \end{aligned} \tag{32}$$

Next, consider a two-input, polynomial time algorithm $A(\cdot)$ that can verify $SATP$. One of the inputs to $A(\cdot)$ is an implementation of constraints (2)-(30). The other input is a binary assignment of the decision variables $\bar{\mathbf{I}}^k_{a,p,l}$, $\bar{\mathbf{T}}^{\text{start}}_{a,p}$, $\bar{\mathbf{D}}_{m,p}$, $\bar{\mathbf{F}}_{m,p}$, and $\bar{\mathbf{T}}^{\text{end}}_{a,p}$. We construct algorithm A as follows: for each instantiation of the constraints, we check to see if the constraints hold in polynomial time. For implementation of constraints (9), (17)-(30), we first convert them to a standard MILP implementation using techniques such as Big-M [27]. The conversion can also be done in polynomial time. The algorithm then finds the maximum value of $\bar{\mathbf{T}}^{\text{end}}_{a,P}$, and checks to see if it is at most T'. This process certainly can be done in polynomial time.□

Proposition 4.2. *The SATP is NP-complete.*

Proof. To prove the SAT is NP-Complete, we must show that it 1) belongs to NP, and 2) is polynomial-time reducible to another problem that is NP-Complete [28]. Proposition 4.1 satisfies the first criterion. For the second

criterion, we will use the Traveling Salesman Problem (TSP), which has been shown to be NP-Complete [28]. We will now show $SATP \leq_P TSP$. Let

$$\begin{aligned} TSP = \{&\langle \mathcal{D}_t, \mathcal{E}_t \rangle, \mathbf{c}_t, T'\} \\ &\mathbf{c}_t : \mathcal{E} \times \mathcal{E} \to \mathbb{R} \\ &T' \in \mathbb{R} \\ &\langle \mathcal{D}_t, \mathcal{E}_t \rangle \text{ has a tour of at most } T_t. \end{aligned} \tag{33}$$

We first construct an algorithm $f_t : TSP \to SATP$ as follows. Since we make no restriction on node $d \in \mathcal{D}_{\mathcal{S}}$ being exclusive to service area $s \in \mathcal{S}$ within SATP we assign the edges of $\mathcal{D}_t$ to map one-to-one to individual service areas, all service areas reachable by service agents and transport agents by all edges, and one of the service areas to serve as the origin node o. Thus, there are $S = |\mathcal{D}_t|$ service areas in the SATP equivalent. The standard TSP problem assumes a route for only one agent. We therefore restrict all mappings of TSP to the case where $N = M = 1$ within the new SATP problem. Likewise, we may consider the 'phases' within the TSP as equal to twice the number of vertices (one each for transit and service), plus two for docking and deployment of the single service agent by the single transport agent. Thus, we have $P = |2\mathcal{D}_t| + 2$. For clarity, we will remove the now superfluous subscripts m and n. Let $c^{\text{dock}} = c^{\text{deploy}} = c^{\text{service}} = 0$ in the new SATP. Let $f_s^{\text{service}} = f_d^{\text{dock}} = f_d^{\text{deploy}} = f_v^{\text{move}} = c^{charge} = 0$. Let $c_{ij}^{\text{move}} = c_{ij}^{trans}$ for all $< i, j >= v \in \mathcal{E}$. These assignments can be done in polynomial time.

We now show that TSP has a solution of at most T' if and only if the mapped SATP has a solution of at most T' as well. Suppose TSP has a solution of at most T'. The assignment of fuel costs to zero causes constraints (8)-(10) to become trivial, and the combination of $c^{\text{dock}} = c^{\text{deploy}} = 0$ and $c_{ij}^{\text{move}} = c_{ij}^{trans}$ cause there to be no difference in servicing time between a service agent moving or being transported and deployed between service areas via (16). The assignment $P = 2|\mathcal{D}_t|+2$ cause the only feasible schedules to be those involving an initial deployment of the single service agent, followed by a TSP tour of alternating service agent transits and servicing actions followed by a single docking, due to constraints (2)-(7) and (30). However, the equal transport/move cost and zero dock and deploy cost assignment causes (31) to degenerate into the simple tour cost of TSP. Thus, the resulting SATP has a minimum time of at most T' as well. Next, suppose that SATP has a completion time of at most T'. Then, since each service node must have been visited exactly once due to the choice of P, and all costs are zero with the

exception of transit costs, the corresponding TSP problem has a tour cost of at most T' as well. $\square$

We now present two strategies for increasing the tractability of the SATP by limiting the required number of variables needed in a particular SATP implementation. The first strategy reduces the number of candidate dock, deploy, and transit nodes to the closest S nodes that provide one node per service area. The second strategy maintains the number of candidate nodes, but eliminates all transition edges between nodes within the same service area. Since the embedded routing problem significantly increases the complexity of scheduling problem as a whole, the general idea of both reduction strategies is to decrease the routing problem in intelligent ways.

4.1. Node Reduction Strategy

We first attempt to reduce the number of variables by selecting only a subset of dock and deploy nodes for consideration in the overall optimization framework, specifically the S nodes in a cluster that minimize the distance traveled to visit all nodes in the subset while providing a dock and deploy point to every service area $\mathcal{S}$. Formally, we can write this node subset as

$$\mathcal{D}' = \left\{ \underset{\mathcal{D}*}{\operatorname{argmin}} \sum_{i,j \in \mathcal{D}^*} c_{ij}^{\text{move}} \,\middle|\, i, j \in \mathcal{D}^*, \mathcal{D}^* \subset \mathcal{D}, \forall s \in \mathcal{S},\, |\mathcal{D}_s^*| = 1 \right\}, \tag{34}$$

where $\mathcal{D}_s^*$ is the set of nodes connected to service areas within the node subset $\mathcal{D}^*$. Equation (34) represents a form of the generalized traveling salesman problem (GTSP) [29, 30]. While the GTSP is another NP-hard problem, its use increases the tractability of the SATP in two ways. First, because the GTSP is partly encapsulated within the SATP when determining routes for the individual agents, solving the GTSP before the main schedule optimization will eliminate orders of magnitude more permutations required in the overall optimization than required to solve the GTSP. Second, there are several heuristics that can quickly provide efficient but sub-optimal solutions to the GTSP [29, 30, 31].

4.2. Edge Reduction Strategy

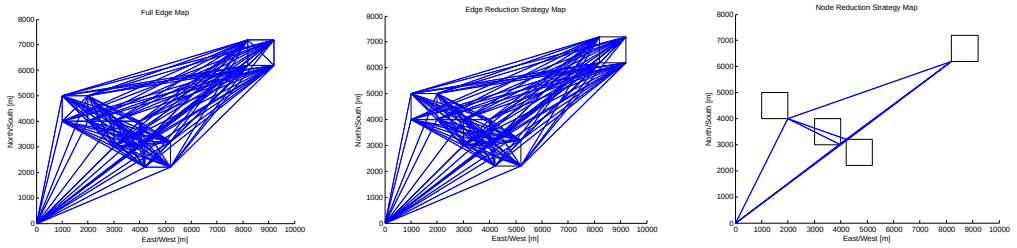
A second candidate strategy for reducing the scale of the SATP is to reduce the number of decision variables by reducing the total number of candidate edges within the problem. Like the node reduction strategy, this is a balancing act between reducing the scale of the problem and eliminating

options for the agents to travel in order to transit in an efficient manner. An appropriate compromise between the two conflicting needs is in the elimination of intra-service area edges. Specifically, we create a new edge map $\mathcal{V}'$ as

$$\mathcal{E}' = \left\{(i,j) \in \mathcal{E} | i \in \mathcal{D}_{s_i}, j \in \mathcal{D}_{s_j}, s_i \neq s_j\right\}. \tag{35}$$

The intuition behind using (35) is the assumption that there is little need for either a service agent or transport agent to transit between two nodes in a single service area, as once an area is serviced, the agents can simply choose the best route to the next service area.

Transit paths between four service areas consisting of four dock and deploy nodes per area are shown in Figure 2. The full graph $\langle \mathcal{D}, \mathcal{E} \rangle$ is shown on the left-hand side of the figure. The graph $\langle \mathcal{D}, \mathcal{E}' \rangle$ resulting from edge reduction strategy is shown in the center. The graph $\langle \mathcal{D}', \mathcal{E} \rangle$ depicting the node reduction strategy is shown on the right. The edge reduction strategy eliminates approximately 18% of the edges in the search space for agent movement. The node reduction dramatically decreases the number of edges and nodes, reducing the number of edges to 8% of the full graph.



(a) Full Graph.

(b) Graph resulting from edge reduction strategy.

(c) Graph resulting from node reduction strategy involving GTSP algorithm.

Figure 2: Comparison between full edge transit graph and graph from complexity reduction strategies. Blue represents edge in transit graph $\mathcal{E}$. Black represents service area boundaries. Left: Complete graph from all dock/deploy points. Center: Graph from edge reduction strategy. Right: Graph from node reduction strategy using the GTSP algorithm.

5. Robust Scheduling Extensions

We now extend our framework to incorporate uncertainty in the duration of various tasks being performed by the individual agents. Our methodol-

ogy is based on the general idea that there is a significant potential cost of schedule slips. Specifically, we define a *schedule slip* as when the end time of a task as scheduled exceeds the scheduled start time of the following task. Often, schedule slips can have catastrophic consequences in real-world multi-agent systems. For example, an agent finishing a service task on time while a transport agent is significantly delayed could loiter in a dangerous location or potentially run out of fuel before docking could be performed. Similarly, a service agent being significantly delayed when expected to rendezvous with a transport agent could significantly delay the entire schedule for the remainder of the mission. In order to develop our robust scheduling scheme to mitigate these hazards with our robust service agent transport problem (R-SATP) extension, we first make the assumption that for actions taken by the individual agents, no tasks can be performed before their scheduled start time. Formally, this can be done by creating a constraint defining the true initial start time of a task as

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p} = \max\left\{\tilde{\mathbf{T}}^{\text{end}}_{a,p-1}, \bar{\mathbf{T}}^{\text{start}}_{a,p}\right\}, \tag{36}$$

where $\tilde{\mathbf{T}}^{\text{end}}_{a,p-1}$ is the true end time of the previous phase for agent a. We then define the true end time of the previous phase as

$$\tilde{\mathbf{T}}^{\text{end}}_{a,p-1} = \tilde{\mathbf{T}}^{\text{start}}_{a,p-1} + \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} \tilde{c}^{k}_{a,p-1,l}\bar{\mathbf{I}}^{k}_{a,p-1,l}, \tag{37}$$

where $\tilde{c}^{k}_{a,p,l}$ is the true duration of task k during phase p performed by agent a on location set l. We now modify the definition of $\bar{\mathbf{T}}^{\text{start}}_{a,p+1}$ from (15) to

$$\bar{\mathbf{T}}^{\text{start}}_{a,p+1} = \bar{\mathbf{T}}^{\text{start}}_{a,p} + \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} c^{k}_{a,l}\bar{\mathbf{I}}^{k}_{a,p,l} + T^{buff}_{a,p}, \tag{38}$$

where $T^{buff}_{a,p}$ is a notional buffer time to prevent schedule slips that we will define with optimization variables subsequently. Beforehand, we will characterize our general strategy of focusing on minimizing schedule slips during individual phases. We do so with the following proposition, which shows that if a phase is currently in a schedule slip situation, there must have been a "root cause" phase in the past where the the time required to perform a task in a phase p' must have exceeded the planned time. We do so, without loss of generality, for the case where $T^{buff}_{a,p} = 0$. In this sense, minimizing the likelihood of schedule slip by creating a statistically optimal buffer time $T^{buff}_{a,p}$ is an appropriate strategy.

Proposition 5.1. *If*

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p} = \tilde{\mathbf{T}}^{\text{end}}_{a,p-1} \tag{39}$$

and

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p} > \bar{\mathbf{T}}^{\text{start}}_{a,p-1} + \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} c^k_{a,l}\bar{\mathbf{I}}^k_{a,p-1,l}, \tag{40}$$

for some $p > 0$, then there exists a $p' \leq p$ such that

$$\sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} \tilde{c}^k_{a,p',l} > \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} c^k_{a,l} \tag{41}$$

Proof. Suppose (39) and (40) are true for some $p > 0$. Since

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p} = \tilde{\mathbf{T}}^{\text{end}}_{a,p-1} \tag{42}$$

$$= \tilde{\mathbf{T}}^{\text{start}}_{a,p-1} + \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} \tilde{c}^k_{a,p-1,l}\bar{\mathbf{I}}^k_{a,p-1,l}, \tag{43}$$

then (40) implies

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-1} + \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} \tilde{c}^k_{a,p-1,l}\bar{\mathbf{I}}^k_{a,p-1,l} > \bar{\mathbf{T}}^{\text{start}}_{a,p-1} + \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} c^k_{a,l}\bar{\mathbf{I}}^k_{a,p-1,l}. \tag{44}$$

There are now two cases for the value of $\tilde{\mathbf{T}}^{\text{start}}_{a,p-1}$.

Case 1:

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-1} = \bar{\mathbf{T}}^{\text{start}}_{a,p-1}. \tag{45}$$

If (45) holds, then from (44) we have

$$\sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} \tilde{c}^k_{a,p',l} > \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} c^k_{a,l} \tag{46}$$

for $p' = p - 1$, and the proposition holds in this case.

In the case that Case 1 does not hold, we have Case 2:

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-1} = \tilde{\mathbf{T}}^{\text{end}}_{a,p-2}. \tag{47}$$

Due to the definition of $\tilde{\mathbf{T}}^{\text{start}}_{a,p}$ and (37),

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-1} = \tilde{\mathbf{T}}^{\text{end}}_{a,p-2} \tag{48}$$

$$= \tilde{\mathbf{T}}^{\text{start}}_{a,p-2} + \sum_{k\in\mathcal{K}}\sum_{l\in\mathcal{L}_k} \tilde{c}^k_{a,p-2,l}\bar{\mathbf{I}}^k_{a,p-2,l}. \tag{49}$$

There are now three possibilities to the relation of $\tilde{\mathbf{T}}^{\text{start}}_{a,p-1}$ to $\bar{\mathbf{T}}^{\text{start}}_{a,p-2} + \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p-2,l}$:

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-1} > \bar{\mathbf{T}}^{\text{start}}_{a,p-2} + \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p-2,l} \tag{50}$$

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-1} < \bar{\mathbf{T}}^{\text{start}}_{a,p-2} + \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p-2,l} \tag{51}$$

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-1} = \bar{\mathbf{T}}^{\text{start}}_{a,p-2} + \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p-2,l}. \tag{52}$$

However, if (51) or (52) hold, it would violate our assumption of being in Case 2. Now by combining equations (48) and (50)

$$\tilde{\mathbf{T}}^{\text{start}}_{a,p-2} + \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} \tilde{c}^k_{a,p-2,l} \bar{\mathbf{I}}^k_{a,p-2,l} > \bar{\mathbf{T}}^{\text{start}}_{a,p-2} + \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p-2,l}. \tag{53}$$

We now have the same assumptions as before for $p-1$. By induction, we can follow the same process to $p = 1$. From initial conditions,

$$\bar{\mathbf{T}}^{\text{start}}_{a,0} = \tilde{\mathbf{T}}^{\text{start}}_{a,0} = 0, \tag{54}$$

which implies

$$\tilde{\mathbf{T}}^{\text{start}}_{a,1} = 0 + \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} \tilde{c}_{a,0,l} \bar{\mathbf{I}}^k_{a,0,l} > \sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,0,l}, \tag{55}$$

satisfying the proposition for $p = 1$.□

We use Proposition 5.1 as motivation to focus on treating the robustification of schedules as a Markov process. Because (41) holds if a schedule slip occurred according to Proposition 5.1, the dominant factor in the probability of schedule slip in the current phase is the uncertainty in the action performed in the previous phase. As a result, we mitigate the possibility of a schedule slip in the overall schedule by reducing the possibility of a schedule slip in every phase. Using (36)-(38), we now characterize our general strategy for reducing the risk of schedule slips.

We assume there is a numerical cost (measured in time) to the occurrence of either a schedule slip, or a delay beyond the buffer time

$$\sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p,l} > T^{buff} \Rightarrow c^{\text{slip}} \tag{56}$$

$$\sum_{k\in\mathcal{K}} \sum_{l\in\mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p,l} < T^{buff} \Rightarrow c^{\text{ineff}}$$

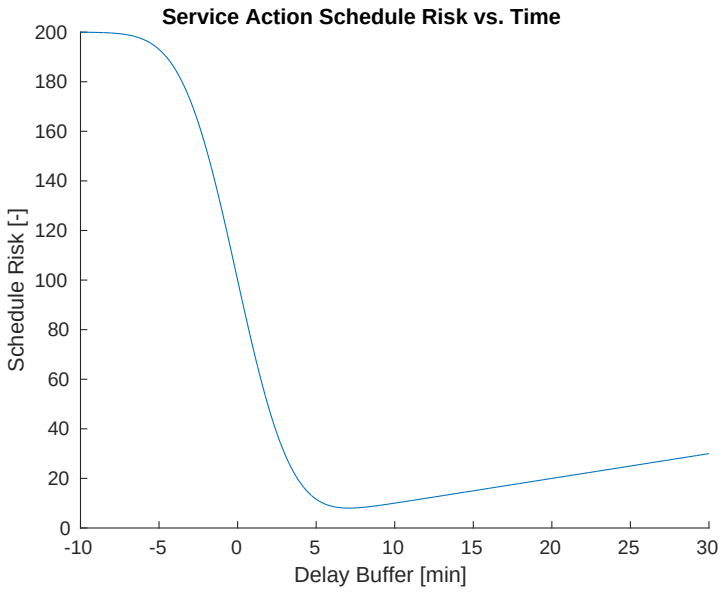


Figure 3: Risk function for Schedule slip based on $\mathcal{N}(0, 20)$ with $c^{\text{ineff}} = T_{a,l}^{k,buff}$ and $c^{\text{slip}} = 200$.

Using the definition of Bayes risk, see [32], we may write the Bayes risk for this cost definition as

$$R = c^{\text{ineff}} P\left(\tilde{c}_{a,l}^{k} < T_{a,l}^{k,buff}\right) P^{\text{ineff}} + c^{\text{slip}} P\left(\tilde{c}_{a,l}^{k} > T_{a,l}^{k,buff}\right) P^{slip}. \tag{57}$$

for a particular action k, where P^{ineff} and P^{slip} are prior probabilities of adverse consequences of either a delay, or a schedule slip, respectively. Varying the buffer time and assuming the task duration follows a $\tilde{c}_{a,l}^{k} \propto \mathcal{N}(0, 20)$ distribution yields a risk function related to buffer time as shown in Figure 3. Note that as written, the risk function is invariant with respect to being performed during a particular phase p. As such, we optimize the nonlinear optimization problem

$$T_{a,l}^{k,buff*} = \underset{T_{a,l}^{k,buff}}{\operatorname{argmin}}\, c^{\text{ineff}} P\left(\tilde{c}_{a,l}^{k} < T_{a,l}^{k,buff}\right) P^{\text{ineff}} + c^{slip} P\left(\tilde{c}_{a,l}^{k} > T_{a,l}^{k,buff}\right) P^{\text{slip}} \tag{58}$$

using numerical techniques prior to the schedule optimization. Once we develop an optimal buffer time for every action, location, and agent (type),

we treat them as parameters in our R-SATP extension. If P^{ineff} and P^{slip} are treated as uniform priors, (58) may be re-written as

$$T_{a,l}^{k,buff*} = \underset{T_{a,l}^{k,buff}}{\operatorname{argmin}} \, c^{\text{ineff}} P\left(\tilde{c}_{a,l}^{k} < T_{a,l}^{k,buff}\right) + c^{\text{slip}} P\left(\tilde{c}_{a,l}^{k} > T_{a,l}^{k,buff}\right). \quad (59)$$

Once the optimal buffer times for each individual task are found using (58), they may be factored into the overall optimization process. In order to incorporate the buffer time, we take advantage of the necessary delays between coordinated agent actions in the SATP by using the wait times as part of the required buffer times. We define the synchronization time for agent a as the time difference between the end time of the current phase and the proceeding phase, written

$$\bar{\mathbf{T}}_{a,p}^{sync} = \bar{\mathbf{T}}_{a,p+1}^{\text{start}} - \bar{\mathbf{T}}_{a,p}^{\text{end}}. \quad (60)$$

We then ensure that the decision variable representing the *scheduled* buffer time $\bar{\mathbf{T}}_{a,p}^{sync}$ is at least the desired, optimal buffer time $T_{a,l}^{k,buff}$ with the constraint

$$\bar{\mathbf{I}}_{a,p,l}^{k} = 1 \wedge \bar{\mathbf{T}}_{a,p}^{sync} \leq T_{a,l}^{k,buff} \Rightarrow \bar{\mathbf{T}}_{a,p}^{buff} = T_{a,l}^{k,buff} - \bar{\mathbf{T}}_{a,p}^{sync}. \quad (61)$$

If the synchronization time is greater than the optimal buffer time parameter $T_{a,l}^{k,buff}$, then the buffer time is set to zero with the constraint

$$\bar{\mathbf{I}}_{a,p,l}^{k} = 1 \wedge \bar{\mathbf{T}}_{a,p}^{sync} \geq T_{a,l}^{k,buff} \Rightarrow \bar{\mathbf{T}}_{a,p}^{buff} = 0 \quad (62)$$

Finally, the robust start time is set as the deterministic start time plus the summation of all buffer times with the constraint

$$\begin{aligned} &\forall a \in \mathcal{A}, \forall p \in \mathcal{P} \\ &\bar{\mathbf{T}}_{a,p+1}^{start,R} = \bar{\mathbf{T}}_{a,p+1}^{\text{start}} + \sum_{p' \in \mathcal{P} s.t. p' \leq p} \bar{\mathbf{T}}_{a,p'}^{buff}. \end{aligned} \quad (63)$$

In order to complete our robust scheduling extension, we modify (16) and

(17) to use the new robust start times, creating the constraints

$$\forall a \in \mathcal{A}, \forall p \in \mathcal{P} \tag{64}$$
$$\bar{\mathbf{T}}^{\text{end}}_{a,p} = \bar{\mathbf{T}}^{\text{start}}_{a,p} + \sum_{k \in \mathcal{K}} \sum_{l \in \mathcal{L}_k} c^k_{a,l} \bar{\mathbf{I}}^k_{a,p,l}$$

$$\forall m \in \mathcal{A}_S, \forall m \in \mathcal{A}_T, \forall p \in \mathcal{P} \tag{65}$$
$$\sum_{d \in \mathcal{D}} \bar{\mathbf{I}}^{\text{dock}}_{mn,p,d} = 1 \Rightarrow \bar{\mathbf{T}}^{\text{start}}_{m,p} = \bar{\mathbf{T}}^{\text{start}}_{n,p}$$

$$\forall m \in \mathcal{A}_S, \forall m \in \mathcal{A}_T, \forall p \in \mathcal{P} \tag{66}$$
$$\sum_{d \in \mathcal{D}} \bar{\mathbf{I}}^{\text{deploy}}_{mn,p,d} = 1 \Rightarrow \bar{\mathbf{T}}^{\text{end}}_{m,p} = \bar{\mathbf{T}}^{\text{end}}_{n,p}.$$

6. Simulation Results

We now present simulation results for the proposed scheduling framework. All simulations were performed using the IBM's ILOG CPLEX optimization software [33]. All simulations were performed on a 2.8 GHz quad-core CPU equipped with 8 GB RAM.

6.1. Deterministic and Robust Scheduling Results

In the following simulations, we assume that a transport agent has a speed of 8 m/s, while service agents have a speed of 1.5 m/s. We assume each of the $S = 4$ service areas requires $c^{\text{service}} = 100$ minutes. Furthermore, we assume that each docking action requires $c^{\text{dock}} = 20$ minutes, and $c^{\text{deploy}} = 10$ minutes. These values are notional times to represent a scheduling scenario. However, they are inspired by our experience in the area of maritime robotics involving unmanned surface vessels and unmanned underwater vehicles [4, 5, 6, 34]. Based on our knowledge of the difficulty with the domain, we believe these values are appropriate for demonstrating our algorithm. Figures 4 and 5 illustrate a simulation of $N = 1$ transport agent, $M = 3$ service agents, and $P = 10$ phases using the node reduction strategy. For the task uncertainty distributions, we used $\sigma^2_{\text{dock}} = 10$, $\sigma^2_{\text{deploy}} = 1$, $\sigma^2_{\text{service}} = 30$, and $\sigma^2_{\text{move}} = \frac{\|<i,j>\|}{10}$ for nodes i and j, respectively. For simulations involving the node reduction strategy, we leverage the algorithm developed by Silberholtz et al. in [31].

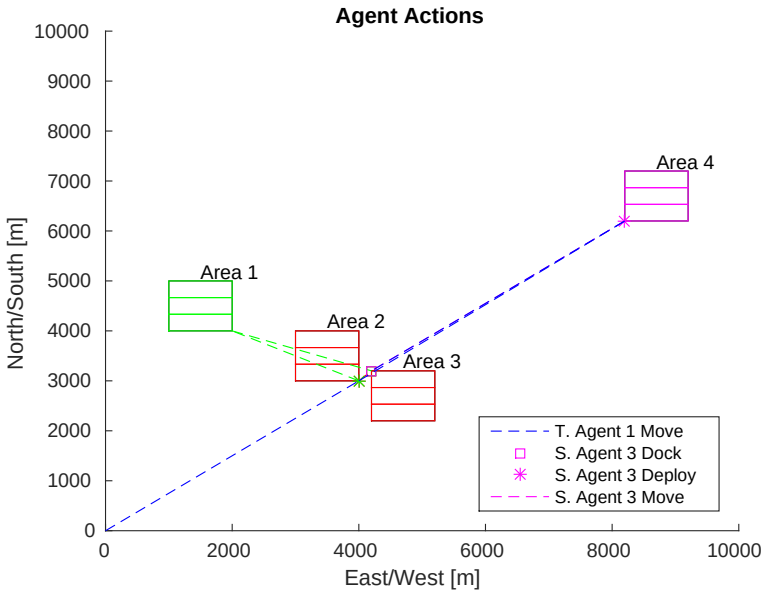


Figure 4: Plot of $A_t = 1$ transport agent and $A_s = 3$ service agent actions. Different colors indicate actions performed by different agents. Blue indicates a transport agent action. Other colors indicate various service agent actions. * indicates service agent deployment points. □ indicates service agent docking points. Dashed lines indicate a move action. Solid line indicates transport action.

Figure 4 represents the spatial map of the various agents moving throughout the field under only the deterministic constraints. The MILP optimization software found a feasible solution within 18% of the optimal solution within one second, and the globally optimal solution in 963.01 seconds, with a maximum end-time objective value of 247.0 minutes. As seen in the figure, the agent schedule has the transport agent deploy service agents 1 and 2 to a deployment node near service areas 2 and 3. Service agent 1 immediately starts servicing the two service areas, while service agent 2 travels to the further service area 1. Simultaneously, the transport agent transits to the furthest service area to deploy service agent 3. The transport agent then returns to a deployment node between service areas 2 and 3 to dock with the three service agents, which transit to the deployment node as they complete their service tasks. From the figure, it is clear that the transport agent takes short, efficient paths to connect all the service areas, and deploys agents where they can have close access to the appropriate service area.

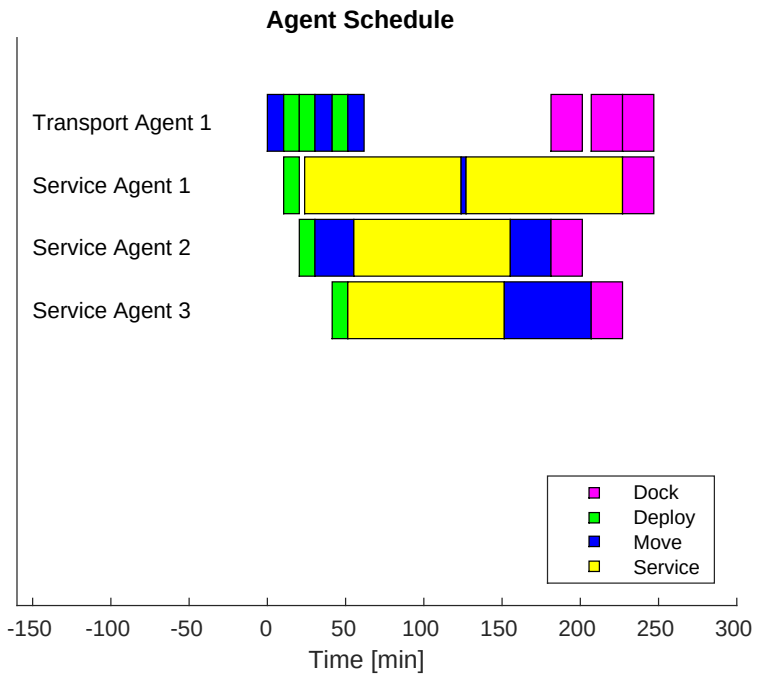


Figure 5: Simulated survey schedule of $A_t = 1$ transport agent and $A_s = 3$ service agents. Blue represents movement actions. Yellow represents service actions. Green and magenta represents deployment and docking actions, respectively.

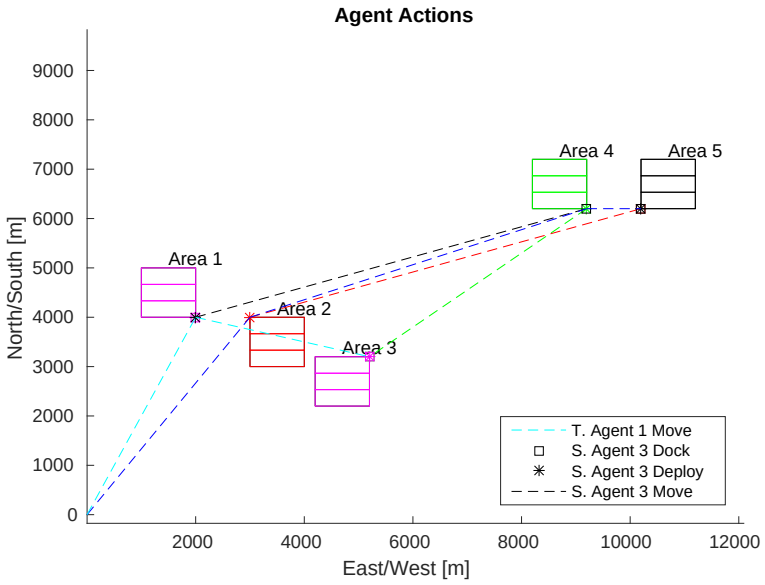


Figure 6: Plot of $A_t = 2$ transport agent and $A_s = 4$ service agent actions. Different colors indicate actions performed by different agents. Blue indicates a transport agent action. Other colors indicate various service agent actions. * indicates service agent deployment points. □ indicates service agent docking points. Dashed lines indicate a move action. Solid line indicates transport action.

Figures 6 and 7 represent a map of agent actions and detailed schedule for $A_s = 4$ and $A_t = 2$ agents prosecuting $S = 6$ service areas using the standard problem without robust optimization. As seen in the figure, the agents are divided among the service areas, with some agents traveling under their own power, and other agents leveraging transport via the transport agents.

We now present a direct comparison of simulations between the deterministic optimization problem, a naive robust optimization problem, and the minimum-risk optimization problem discussed in Section 5 for $A_t = 1$ transport agent and $A_s = 2$ survey agents within $P = 12$ phases. A naive method for mitigating concerns of schedule slips is typically to add a fixed amount of buffer time, often as a function of the standard deviations. As such, to compare the deterministic method to our robust counterpart, we provide a baseline robust simulation where three standard deviations of time required are added for each action type. For example, a service action could be planned for $c^{\text{service}} = 116.425$ seconds. For the minimum-risk optimization problem, we set $c^{\text{slip}}_{a,l} = 500$ and $c^{\text{ineff}} = T^{buff}_{a,l}$ for all task types.

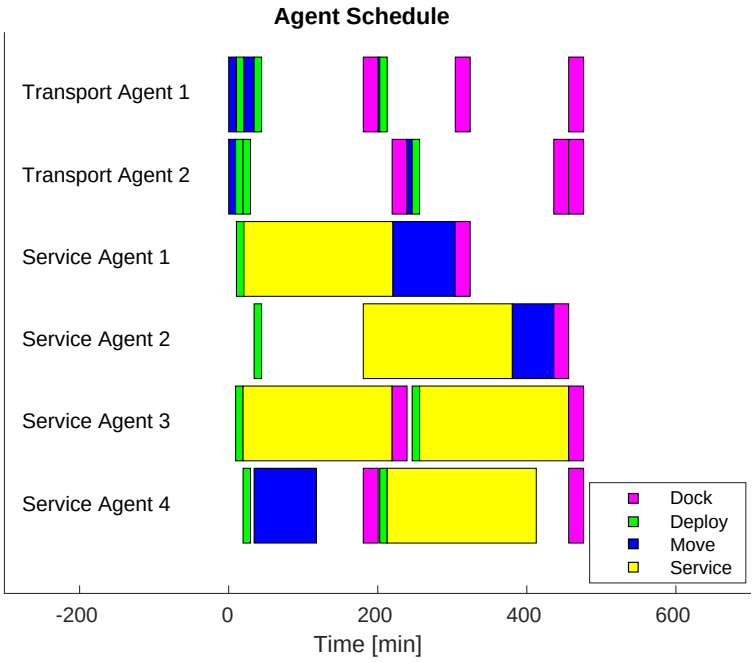


Figure 7: Simulated survey schedule of $A_t = 2$ transport agent and $A_s = 4$ service agent actions. Blue represents movement actions. Yellow represents service actions. Green and magenta represents deployment and docking actions, respectively.

Figure 8 shows a comparison using the three different optimization problems. As expected, the deterministic solution has the best objective vale (completion time) of 310.9 minutes. The naive robust solution had a completion time of 364.9 minutes. The minimum-risk robust scheduling solution had a completion time of 345.45 minutes. Also of note is the different strategies taken by the deterministic or naive solution, and the minimum-risk solution. The deterministic and naive robust solutions both schedule the transport agent to pick up and then re-deploy one of the service agents to the two furthest service areas. The minimum-risk scheduling algorithm has the transport agents both deployed at the closest service areas, and one travels to each of the furthest areas under its own power. We conjecture that this is likely due to the minimum-risk scheduling taking into account the increased schedule slip risk due to executing an increased number of tasks required for dock and re-deployment versus a single move action. We empirically analyze the computational performance of the minimum-risk robust solution versus the deterministic solution in the following section.

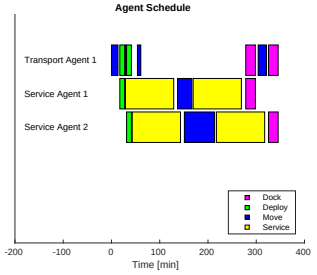
(a) Deterministic solution action plot.

(b) Naive robust solution action plot.

(c) Minimum-risk robust solution action plot.

(d) Deterministic solution schedule.

(e) Naive robust solution schedule.



(f) Minimum-risk robust solution schedule.

Figure 8: Comparison between deterministic optimization, a naive robust optimization approach, and the proposed minimum-risk scheduling approach.

actions action.
assume that dock/deploy point

6.2. Analysis of Complexity Reduction Heuristics and Computational Tractability

Figure 9 shows results from a Monte Carlo simulation comparing the full graph to the two complexity reduction strategies discussed in Section 4 for a deterministic SATP optimization problem involving $N = 1$ transport agent, $M = 3$ service agents, and $P = 15$ phases. Sixty simulations were performed with three service areas randomly generated over a 10,000m by 10,000m field. For each simulation, the cutoff time for providing the best solution from CPLEX for the optimization problem was 120 seconds. As seen in the figure, the node reduction strategy resulted in the fastest sortie time, averaging an objective value (31) of 281.4 minutes. The edge-reduction strategy provided a modest improvement over the full optimization model, averaging an objective value of 373.1. As expected, the full optimization model, due to its significantly greater complexity, resulted in the poorest solutions, averaging 398.8 minutes. Additionally, in some instances, the optimization solver was unable to find a solution to the problem in the specified time limit due to the computational complexity of the full optimization problem. The timeout without a solution occurred in 48% of the full optimization problem's simulations, and 23% of simulations using the edge reduction strategy. However, in no instances of using the node reduction strategy did a timeout occur.

A second Monte Carlo simulation was performed involving sixty simulations of a single transport agent and service agent, and $P = 12$ phases, allowing all optimization routines to run until a globally optimal solution is found. In this manner, the Monte Carlo simulation characterized how the two heuristic reduction strategies (34) and (35) impact the discovery of the globally optimal schedule by eliminating transit paths. In all cases, the globally optimal solution to the full optimization problem was identical to the globally optimal solution of the edge reduction strategy (35). The node reduction strategy (34) did eliminate the globally optimal solution in all instances, however this elimination increased the minimum solution's objective function value by an average of only 3.45%, and maximum of 7.7%.

In addition to the simulations characterizing the effects of the various heuristics on the quality of the optimization result, we characterize the effect of increasing number of (binary) optimization variables on the optimization

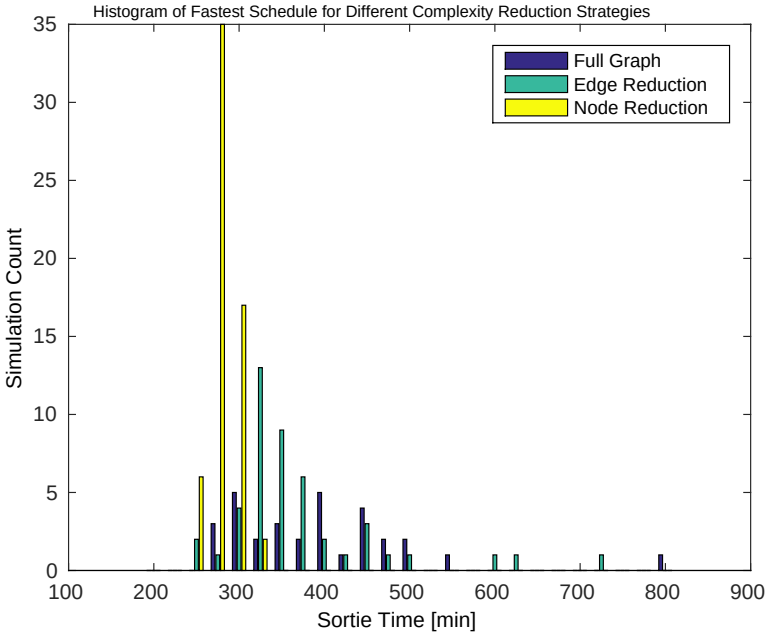


Figure 9: Histogram of Monte Carlo simulations showing minimum objective value found by different complexity reduction strategies.

framework while using the node reduction heuristic. Table 1 shows a number of simulations that vary the agent count and number of phases in which to schedule tasks. Additionally, the simulations show the difference between the robust versus deterministic SATP case, and the result of the optimization yielding the best solution from CPLEX after 300 seconds of run-time. If computational tractibility were irrelevant, the best solution would be found from the problem using the most phases and agents, since the schedule would take advantage of tasking multiple agents, and slotting those agents in complicated task schedules. Asterisks represent where the globally optimal schedule was found within the cutoff time. For $A \geq 5$ agents, two transport agents were used in the scenario. As seen in the table, the lower number of phases/agents resulted in a higher likelihood of the globally optimal schedule being found. As a larger number of agents and phases were used, the results were generally sub-optimal, and at times a worse result than a fewer number of phases for the same agent number, due to the dramatically increased search space due to the binary variables. From the table and judging by the variation in best search schedules returned before the cut-off time, reasonable numbers of agents for the SATP scheduling problem as developed appears to be four agents or fewer for the deterministic schedule, and three agents and fewer than $P = 14$ phases for the robust scheduling extension. The only scenario that failed to produce a result was the most complicated of six agents and 20 phases using the robust extension, which created an optimization problem of 26,190 binary variables. This compares to the simplest scenario, which created an optimization problem of 869 binary variables.

7. Discussion

As shown in the analysis and simulations, the computational tractability of the SATP is a key concern with the general problem. While our particular solution attempts to reduce the search space by eliminating poor choices using the node and edge reduction strategies, the strategies are only effective for a moderate number of agents before the optimization search space becomes intractable despite the strategies. A second challenge with the current framework is determining the appropriate number of phases P. Too few phases, and a problem may be infeasible. Too many, and the problem can be computationally intractable Developing alternative optimization frameworks that allow for increased computational efficiency is a potential area of future exploration.

Table 1: Characterization of phase, agent count, and optimization type effects on optimization quality.

Agent Count	**Opt. Type**	**# of Phases**				
		12	14	16	18	20
2	Standard	522.86*	507.72*	507.72*	507.72*	507.72*
	Robust	567.61	559.12	567.61	567.61	567.61
3	Standard	297.72	295.16	303.57	303.57	303.57
	Robust	337.12	338.01	349.68	360.39	384.61
4	Standard	244.14	244.14	244.14	243.55	244.14
	Robust	377.62	314.24	333.70	373.47	390.96
5	Standard	271.01	271.01	281.41	296.43	275.08
	Robust	313.44	513.45	389.08	409.80	277.45
6	Standard	173.02	188.44	236.97	171.41	192.42
	Robust	424.41	379.24	275.37	435.07	—

One of the most challenging aspects of the work was in the level of detail required to appropriately model the SATP within a MILP framework. Many of the constraints were only determined to be necessary after initial development of the optimization model and iteration due to obtaining solutions that were unrealistic for real agents. An example of this was in some of the detailed docking and deployment constraints such as (23). After simulations with our initial model, we noticed solutions returned where an agent was deployed from a service area in which it was not located. While MILP optimization is a powerful tool, neglecting subtle details in the model development may produce unrealistic solutions, and thus should be used carefully. Another area of potential future exploration is in multivariate optimization. The SATP as presented only attempts to minimize completion time. However, with multiple agents performing a complicated task there is the tendency to return some solutions that, while technically optimal and sufficient, have features that would not be desired in a real-world setting. An example of this is in the results shown in Figures 6 and 7. In the simulation, several service agents take long transits on their own power. While this is a feasible solution, an equally feasible and more desireable solution could be to use a transport agent if it does not increase mission execution time. Balancing these real-world preferences would be a valuable extension of this work.

The robust scheduling extensions provide a formal method to incorporate both the likelihood of a schedule slip as well as any costs associated with a

schedule slip occurring. Two key challenges with the method, however, include appropriately choosing the costs associated with schedule slip and the increased computational complexity of the MILP optimization framework using the method. While Bayes risk allows a user to tune the buffer decision more accurately than a simple fixed amount, it depends on the user or application knowing what the fixed cost of schedule slip is. In our work we simply assign a fixed value as a representative example. In more realistic or applied scenarios, the costs must be determined in order to appropriately balance the risk of schedule slips to the delay from planned buffer times that ensure a schedule slip will not occur.

8. Conclusions

We have presented a novel problem for planning the task allocation and schedules for service agents that directly affect a number of service areas and transport agents to act as transportation between areas. We also presented a framework for developing efficient schedules for the heterogeneous team using mixed integer linear programming and complexity reduction techniques. This provides an initial step in formally planning service agent - transport agent cooperation. Our technique allows for the formal modeling of the required constraints to optimize both the standard problem and a robust solution variant.

Future work includes developing a decentralized optimization process to make the scheduling algorithms more computationally tractable for larger numbers of agents. While the proposed approach is reasonable for a small team of service agents and transport agents, attempting to plan a large-scale plan of tens or even hundreds of agents appears to be currently infeasible. A decentralize approach may allow the problem to be solved for large teams of agents.

9. Acknowledgments

This work has been supported by the Office of Naval Research (ONR) Code 32 and ONR Independent Applied Research programs.

References

[1] A. J. Shafer, M. R. Benjamin, J. J. Leonard, J. Curcio, Autonomous cooperation of heterogeneous platforms for sea-based search tasks, in: MTS/IEEE OCEANS, 2008.

[2] E. G. Jones, B. Browning, M. B. Dias, B. Argall, M. M. Veloso, Dynamically formed heterogeneous robot teams performing tightly coordinated tasks, in: IEEE International Conference on Robotics and Automation, 2006.

[3] T. Estlin, D. Gaines, F. Fisher, R. Castano, Coordinating multiple rovers with interdependent science objectives, in: Proceedings of the fourth international joint conference on autonomous agents and multiagent systems, 2005.

[4] P. Sujit, A. Healey, J. B. Sousa, AUV docking on a moving submarine using a K-R navigation function, in: International Conference on Intelligent Robots and Systems, 2011.

[5] B. W. Hobson, R. S. McEwen, J. Erickson, T. Hoover, L. McBride, F. Shane, J. G. Bellingham, The development and ocean testing of an AUV docking station for a 21" AUV, in: IEEE/MTS OCEANS, 2007.

[6] A. Martins, J. M. Almeida, H. Ferreira, H. Silva, N. Dias, A. Dias, C. Almeida, E. Silva, Autonomous surface vehicle docking manoevre with visual information, in: International Conference on Robotics and Automation, 2007.

[7] J. N. Weaver, D. Z. Frank, E. M. Swartz, A. A. Arroyo, UAV performing autonomous landing on USV utilizing the robot operating system, in: ASME Early Career Technical Symposium, 2013.

[8] K. E. Wenzel, A. Masselli, A. Zell, Automatic takeoff, tracking and landing of a miniature UAV on a moving carrier vehicle, Journal of Intelligent Robot Systems 61 (2011) 221–238.

[9] D. Bamburry, Drones: Designed for product delivery, Design Management Review 26 (1) (2015) 40–48. doi:10.1111/drev.10313.
URL `http://dx.doi.org/10.1111/drev.10313`

[10] E. I. Sarda, M. R. Dhanak, A usv-based automated launch and recovery system for auvs, IEEE Journal of Oceanic Engineering PP (99) (2016) 1–19. doi:10.1109/JOE.2016.2554679.

[11] C. A. Mendez, J. Cerda, An milp framework for batch reactive scheduling with limited discrete resources, Computers & Chemical Engineering.

[12] X. Lin, S. L. Janak, C. Floudas, A new robust optimization approach for scheduling under uncertainty: I. Bounded uncertainty, Computers & Chemical Engineering 29 (2003) 1069–1085.

[13] M. C. Gombolay, R. J. Wilcox, J. A. Shah, Fast scheduling of multi-robot teams with temporospatial constraints, in: Robotics: Science and Systems, 2013.

[14] O. Kone, C. Artigues, P. Lopes, M. Mongeau, Comparison of mixed intiger linear programming models for the resource-constrained project scheduling problem with consumption and production of resources, Flexible Services and Manufacturing Journal.

[15] M. Koes, I. Nourbakhsh, K. Sycara, Constraint optimization coordination architecture for search and rescue robotics, in: International Conference on Robotics and Automation, 2006.

[16] G. A. Korsah, A. Stentz, M. B. Dias, I. Fanaswala, Optimal vehicle routing and scheduling with precedence constraints and location choice, in: International Conference on Robotics and Automation, 2010.

[17] C. Mandalandraki, M. S. Daskin, Time dependent vehicle routing problems: Formulations, properties, and heuristic algorithms, Transportation Science 26 (1992) 185–200.

[18] N. Mathew, S. L. Smith, S. L. Waslander, Optimal path planning in cooperative heterogeneous multi-robot delivery systems, in: Workshop on the Algorithmic Foundations of Robotics, 2014.

[19] E. Garone, J.-F. Determe, R. Naldi, A travelling salesman problem for a class of heterogeneous multi-vehicle systems, in: 2012 IEEE 51st IEEE Conference on Decision and Control (CDC), IEEE, 2012, pp. 1166–1171.

[20] R. Dechter, Temporal constraint networks, Artificial Intelligence 49 (1991) 61–95.

[21] S. L. Janak, X. Lin, C. Floudas, A new robust optimization approach for scheduling under uncertainty: Ii. uncertainty with known probability distribution, Computers & Chemical Engineering 31 (2006) 171–195.

[22] D. Bertsimas, M. Sim, The price of robustness, Operations Research 52 (1) (2004) 35–53.

[23] R. L. Daniels, P. Kouvelis, Robust scheduling to hedge against processing time uncertainty in single-stage production, Management Science 41 (2) (1995) 363–376.

[24] M. Lombardi, M. Milano, L. Benini, Robust scheduling of task graphs under execution of time uncertainty, IEEE Transactions on Computers.

[25] A. J. Davenport, C. Gefflot, J. C. Beck, Slack-based techniques for robust schedules, in: Sixth European Conference on Planning, 2014.

[26] S. D. Wu, E.-S. Byeon, R. H. Storer, A graph-theoretic decomposition of the job shop scheduling problem to achieve scheduling robustness, Operations Research 47 (1) (1999) 113–124.

[27] A. Bemporad, M. Morari, Control of systems integrating logic, dynamics, and constraints, Automatica 35 (3) (1999) 407 – 427. doi:DOI: 10.1016/S0005-1098(98)00178-2.

[28] T. . Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, Introduction to Algorithms, The MIT Press, 2009.

[29] L. Davis, Applying adaptive algorithms to epistatic domains, in: International Joint Conference on Artificial Intelligence, 1985.

[30] M. Fichetti, J. J. S. Gonzalez, P. Toth, A branch and cut algorithm for the symmetric generalized traveling salesman problem, Operations Research 45 (1997) 378–394.

[31] J. Silberholz, B. Golden, The generalized traveling salesman problem: A new genetic algorithm approach, Extended Horizons: Advances in Computing, Optimization, and Decision Technologies 37 (2007) 165–181.

[32] H. V. Poor, An Introduction to Signal Detection and Estimation, Springer, 1994.

[33] IBM Corporation, IBM ILOG CPLEX User Manual (2013).

[34] M. J. Bays, R. D. Tatum, L. Cofer, J. R. Perkins, Automated scheduling and mission visualization for mine countermeasure operations, in: OCEANS 2015-MTS/IEEE Washington, IEEE, 2015, pp. 1–7.